Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

**Лабораторная работа №5**

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

**Выполнил:**

Студент 3 курса

Группы ИИ-26(1)

Пасевич К.Ю.

**Проверила:**

Андренко К.В.

Брест 2026

**Цель работы:** реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

**Вариант:** тот же, что в Лабораторной №4 (логическая функция и значение n).

**Ход работы**

1. Сгенерировать полную таблицу истинности для своей логической функции ($2^n$ примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте np.random.seed(42) для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:

- Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
- Реализовать правильный градиент для BCE + сигмоида:

4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):

А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)

Б. MSE + адаптивный шаг (из ЛР №4)

В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)

Г. BCE + адаптивный шаг (по формуле из ЛР №2)

Критерий остановки для всех – суммарная ошибка Es ≤ Ee (возьмите Ee из ЛР №2 или №3, например 0.05 или 0.01).

5. Для каждой конфигурации:

- Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
- Зафиксировать количество эпох до достижения критерия остановки.
- После обучения посчитать accuracy на обучающей и тестовой выборках.
- Проверить, сколько процентов из всей таблицы истинности ($2^n$ примеров) сеть предсказывает правильно.

6. Построить один общий график зависимости суммарной ошибки Es от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).

7. Реализовать режим функционирования сети:

- Пользователь вводит вектор из n значений (0 или 1).
- Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).

- Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».

8. Написать вывод (обязательно ответить на следующие пункты):

- Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
- Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
- Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?
- Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
- Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

***Код программы:***

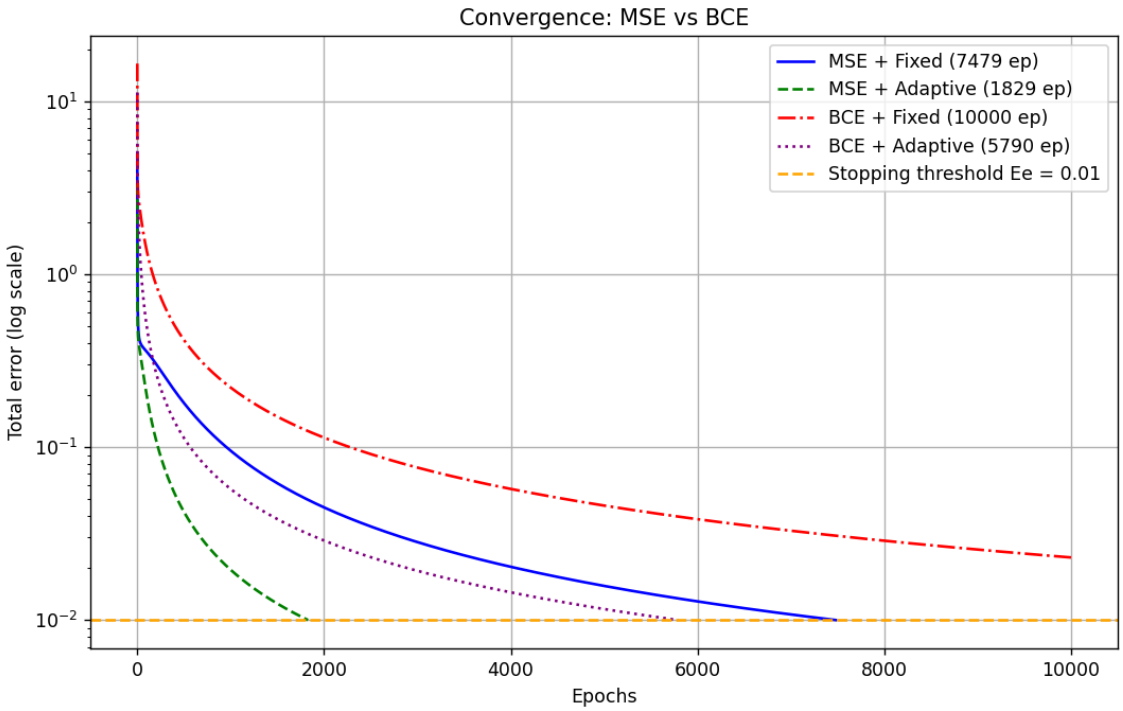
```
import numpy as np
import matplotlib.pyplot as plt
import itertools

# 1. Generate full truth table (2^n examples)
num_inputs = 7
stop_error_threshold = 0.01
fixed_learning_rate = 0.1
np.random.seed(42)

full_input_matrix = np.array(list(itertools.product([0, 1],
repeat=num_inputs)))
full_output_vector = np.array([0 if np.all(sample == 0) else 1 for sample in
full_input_matrix])

# 2. Split data into training (80%) and test (20%)
zero_class_indices = np.where(full_output_vector == 0)[0]
one_class_indices = np.where(full_output_vector == 1)[0]

np.random.seed(42)
np.random.shuffle(zero_class_indices)
np.random.shuffle(one_class_indices)

train_zero_count = 1
train_one_count = int(0.8 * len(full_input_matrix)) - 1

train_indices = np.concatenate((zero_class_indices[:train_zero_count],
one_class_indices[:train_one_count]))
test_indices = one_class_indices[train_one_count:]

np.random.shuffle(train_indices)
np.random.shuffle(test_indices)

train_inputs, train_outputs = full_input_matrix[train_indices],
full_output_vector[train_indices]
test_inputs, test_outputs = full_input_matrix[test_indices],
full_output_vector[test_indices]
```

```
train_inputs_with_bias = np.c_[np.ones(len(train_inputs)), train_inputs]
test_inputs_with_bias = np.c_[np.ones(len(test_inputs)), test_inputs]
full_inputs_with_bias = np.c_[np.ones(len(full_input_matrix)),
full_input_matrix]

# Sigmoid activation function
def sigmoid_activation(linear_output):
    return 1 / (1 + np.exp(-np.clip(linear_output, -100, 100)))

# 3. Perceptron training with 4 configurations (online mode)
def train_perceptron_model(input_matrix, target_outputs, loss_function_type,
learning_rate_mode):
    np.random.seed(42)
    weight_vector = np.random.uniform(-0.1, 0.1, num_inputs + 1)
    error_history_per_epoch = []

    for epoch_idx in range(10000):
        total_epoch_error = 0
        for sample_idx in range(len(input_matrix)):
            current_input = input_matrix[sample_idx]
            desired_output = target_outputs[sample_idx]

            predicted_output = sigmoid_activation(np.dot(weight_vector,
current_input))

            # Fixed or adaptive learning rate
            if learning_rate_mode == 'fixed':
                current_learning_rate = fixed_learning_rate
            else:
                current_learning_rate = 1.0 / (1.0 + np.sum(current_input **
2))

            # MSE or BCE loss with correct gradient
            if loss_function_type == 'MSE':
                total_epoch_error += 0.5 * (desired_output -
predicted_output) ** 2
                gradient = (desired_output - predicted_output) *
predicted_output * (1 - predicted_output)
            else:
                epsilon = 1e-15
                clipped_prediction = np.clip(predicted_output, epsilon, 1 -
epsilon)
                total_epoch_error += - (desired_output *
np.log(clipped_prediction) + (1 - desired_output) * np.log(1 -
clipped_prediction))
                gradient = (desired_output - predicted_output)

            weight_vector += current_learning_rate * gradient * current_input

        error_history_per_epoch.append(total_epoch_error)
        # Stopping criterion: total error <= stop_error_threshold
        if total_epoch_error <= stop_error_threshold:
            break

    return weight_vector, error_history_per_epoch, epoch_idx + 1

# 4. Four training configurations
training_configs = {
```

```
    'MSE + Fixed': ('MSE', 'fixed'),
    'MSE + Adaptive': ('MSE', 'adaptive'),
    'BCE + Fixed': ('BCE', 'fixed'),
    'BCE + Adaptive': ('BCE', 'adaptive')
}

training_results = {}
for config_name, (loss_func, lr_mode) in training_configs.items():
    training_results[config_name] =
train_perceptron_model(train_inputs_with_bias, train_outputs, loss_func,
lr_mode)
    print(f"{config_name} completed. Epochs:
{training_results[config_name][2]}")

# 5. Plot convergence graph (4 curves, different colors)
plt.figure(figsize=(10, 6))
curve_colors = ['blue', 'green', 'red', 'purple']
line_styles = ['-', '--', '-.', ':']
for (config_name, (final_weights, error_history, total_epochs)), line_color,
line_style in zip(training_results.items(), curve_colors, line_styles):
    plt.plot(error_history, label=f"{config_name} ({total_epochs} ep)",
color=line_color, linestyle=line_style)

plt.axhline(y=stop_error_threshold, color='orange', linestyle='--',
label=f'Stopping threshold Ee = {stop_error_threshold}')
plt.yscale('log')
plt.title('Convergence: MSE vs BCE')
plt.xlabel('Epochs')
plt.ylabel('Total error (log scale)')
plt.legend()
plt.grid(True)
plt.show()

# 6. Calculate accuracy on train, test, and full truth table
def compute_accuracy(model_weights, input_matrix, true_outputs):
    predictions = (sigmoid_activation(np.dot(input_matrix, model_weights)) >=
0.5).astype(int)
    return np.mean(predictions == true_outputs) * 100

print("\nResults:")
for config_name, (final_weights, error_history, total_epochs) in
training_results.items():
    train_accuracy = compute_accuracy(final_weights, train_inputs_with_bias,
train_outputs)
    test_accuracy = compute_accuracy(final_weights, test_inputs_with_bias,
test_outputs)
    full_accuracy = compute_accuracy(final_weights, full_inputs_with_bias,
full_output_vector)
    print(f"{config_name:<15} | Ep: {total_epochs:<5} | Train:
{train_accuracy:>5.1f}% | Test: {test_accuracy:>5.1f}% | Full:
{full_accuracy:>5.1f}%")

# 7. Operation mode: user input vector, output probability and class
print("\nOperation mode:")
final_model_weights = training_results['BCE + Adaptive'][0]
try:
    user_input_str = input("Enter vector (7 bits separated by spaces): ")
    user_input_vector = np.array([int(bit) for bit in
user_input_str.split()])
```

```
    user_input_with_bias = np.insert(user_input_vector, 0, 1)
    output_probability = sigmoid_activation(np.dot(final_model_weights,
user_input_with_bias))
    predicted_class = 1 if output_probability >= 0.5 else 0
    true_class_for_or = 0 if np.all(user_input_vector == 0) else 1
    match_status = "Matches truth table" if predicted_class ==
true_class_for_or else "Mismatch"
    print(f"Probability ŷ: {output_probability:.4f}")
    print(f"Predicted class: {predicted_class}")
    print(match_status)
except Exception:
    pass
```

***Результат программы:***




MSE + Fixed completed. Epochs: 7479

MSE + Adaptive completed. Epochs: 1829

BCE + Fixed completed. Epochs: 10000

BCE + Adaptive completed. Epochs: 5790

Results:

MSE + Fixed | Ep: 7479 | Train: 100.0% | Test: 100.0% | Full: 100.0%

MSE + Adaptive | Ep: 1829 | Train: 100.0% | Test: 100.0% | Full: 100.0%

BCE + Fixed | Ep: 10000 | Train: 100.0% | Test: 100.0% | Full: 100.0%

BCE + Adaptive | Ep: 5790 | Train: 100.0% | Test: 100.0% | Full: 100.0%

Operation mode:

Enter vector (7 bits separated by spaces): 0 0 0 0 0 0 0

Probability ŷ: 0.0041

Predicted class: 0

**Итоги по результатам лабораторной работы**

1. Какой из методов сходится быстрее: MSE или BCE?

По результатам обучения видно, что MSE сходится значительно быстрее BCE. При фиксированном шаге MSE достигла порога за 7479 эпох, а BCE за 10000 эпох так и не смогла его достичь: обучение остановилось по лимиту. При адаптивном шаге MSE обучилась за 1829 эпох, а BCE: за 5790 эпох. То есть BCE замедляет обучение примерно в 3 раза по сравнению с MSE при использовании адаптивного шага.

2. Почему BCE теоретически лучше подходит для классификации, чем MSE?

Несмотря на то, что в данном эксперименте MSE обучилась быстрее, теоретически BCE считается более подходящей для задач классификации. Главная причина: проблема затухающего градиента у MSE. Когда сеть делает уверенное, но неправильное предсказание (например, должен быть единица, а сеть выдаёт 0.01), градиент MSE умножается на производную сигмоиды, которая в этом случае почти равна нулю. Из-за этого сеть практически перестаёт обучаться и не может исправить свою ошибку. У BCE такого множителя нет: градиент остаётся большим, и сеть продолжает учиться даже на уверенно неправильных ответах. Кроме того, BCE имеет чёткую вероятностную интерпретацию: она минимизирует расхождение между предсказанным распределением вероятностей и истинным. Это делает её естественным выбором для бинарной классификации.

3. Как изменилась обобщающая способность при переходе на BCE?

В данном эксперименте все четыре конфигурации показали 100% точность на обучающей, тестовой и полной выборках. То есть разницы в обобщающей способности между MSE и BCE не наблюдается. Это объясняется тем, что задача OR с семью входами является линейно разделимой и достаточно простой. На более сложных задачах, особенно с несбалансированными классами или с уверенно неправильными предсказаниями, BCE обычно показывает лучшую обобщающую способность.

4. Как влияет адаптивный шаг на обучение с BCE?

Адаптивный шаг оказывает критически важное влияние на обучение с BCE. При фиксированном шаге BCE не смогла достичь порога остановки за 10000 эпох: обучение прервалось по лимиту. А с адаптивным шагом BCE успешно достигла порога за 5790 эпох. То есть адаптивный шаг ускорил обучение BCE более чем в 1.7 раза и, что ещё важнее, сделал его возможным. Формула адаптивного шага, которая учитывает норму входного вектора, автоматически подбирает оптимальную величину обновления весов для каждого примера, что особенно полезно для BCE.

5. Достаточно ли однослойного персептрона для логических функций?

Однослойного персептрона достаточно только для линейно разделимых логических функций. К таким функциям относятся AND, OR, NAND, NOR: и именно с такими функциями однослойный персептрон справляется идеально, что и подтверждается 100% точностью в данном эксперименте. Однако для нелинейно разделимых функций, например для XOR (исключающего ИЛИ), одного слоя недостаточно. XOR нельзя разделить одной прямой линией на плоскости: для этого нужна хотя бы один скрытый слой нейронов. Для более сложных реальных задач, таких как распознавание рукописных цифр, изображений

или речи, требуются многослойные нейронные сети с несколькими скрытыми слоями и нелинейными функциями активации.

**Вывод:** реализовала однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провела сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовала влияние адаптивного шага обучения на BCE.